Universal Adversarial Trigger Algorithm for Protein Folding

Harys Dalvi
Department of Computer Science
Brown University
Providence, RI 02912
hdalvi@cs.brown.edu

Nitin Sreekumar
Department of Computer Science
Brown University
Providence, RI 02912
nitin_sreekumar@brown.edu

Abstract

In natural language processing, many large language models show greatly reduced performance on a prompt if the prompt is preceded or followed by particular triggers. Given access to the model, these triggers can be discovered through a white-box search. In this paper, we extend the algorithm for discovering such triggers from natural language processing to the domain of protein folding. We run the algorithm in an adversarial setting to increase the model's error, as well as the reverse to discover triggers that decrease the model's error. We also repurpose this algorithm for the problem of inverse folding: predicting a protein's sequence given its structure. While the adversarial and inverse folding settings were unsuccessful, we discovered a trigger that spuriously increases prediction accuracy in ESMFold despite making the sequence less similar from the ground truth. We suggest a number of applications and future directions based on this research.

1 Introduction

1.1 Protein Folding

Ever since the discovery of the first protein structure in 1958, the accurate prediction of protein structures has been a focus point for computational biologists. After all, proteins are considered the functional component of life. They are responsible for nearly every task a cell completes, ranging from facilitating chemical reactions to providing the cellular structure to regulating feedback loops. The function of a protein is determined by the three-dimensional structure, meaning that knowing the structure of a given protein provides insight into its function in the cell.

Modern protein folding studies are rooted in Anfinsen's initial studies in the 1970's which Anfinsen's hypothesis. This thermodynamics-based theory demonstrated that the structure a protein forms in native conditions is based on the amino acid sequence and is the structure that is the most thermodynamically stable. This discovery established that each protein has a unique structure determined by the amino acid sequence that comprises its primary structure and indicated that it is possible to predict the structures of proteins.[1]

In the years since, many methods have been utilized to discover existing protein structures. Experimental methods such as X-ray crystallography, nuclear magnetic resonance (NMR), and cryo-EM are fairly accurate but require a lot of time, labor, and money. Further, not all proteins can undergo the preparation needed to utilize these techniques, reducing their usefulness even more. In fact, out of the around 180 million proteins in the Universal Protein database,[11] only 170 thousand structures have been found experimentally using these methods.[4]

Computational methods have also been attempted. Molecular dynamics approaches attack the problem by simulating how all the atoms in a molecule interact with each other to provide a physics-based

Preprint. Under review.

model of protein folding. While such simulations provide high-resolution data on folding processes, they must be run for a long time and require large amounts of computing resources to model the thousands of atoms in an average protein as well as the surrounding water molecules which also play a role.[6][14]

Recently, however, machine learning techniques have been applied to this protein folding problem with great success. One of the first models to rise to tackle the protein folding problem is AlphaFold,[7] which uses multiple sequence alignment (MSA) and homologous templates to produce predicted protein structures based on real-world structures. Another popular and successful model is ESMFold[9] which uses a similar architecture to transformers, originally designed for natural language processing.[12] This allows it to treat protein sequences as a language rather than relying on MSAs, and it computes predictions faster with similar accuracy.

1.2 Adversarial Triggers

One of the more studied aspects of machine learning is Natural Language Processing (NLP), and the rapid advancements in NLP have led models to see much success in chatbots such as ChatGPT. However, there are still many ways to manipulate these models. One way is through adversarial triggers, which are short, input-agonistic series of tokens that when added to an input can meaningfully change the NLP model's response. These sequences can be found using a gradient descent optimization algorithm and were able to modify outputs to contain racist or offensive text regardless of the input as long as the trigger was combined.

As ESMFold is derived from NLP, it could possibly be susceptible to adversarial triggers. Adding a short amino acid sequence to the end of a protein sequence could exploit biases in the learning process and drastically change the protein shape and thus its functional attributes. This can have important implications when these protein folding models are used for drug discovery and disease characterization. By knowing more about how the model reacts to these triggers, we can ensure it works properly when they are utilized in research. In this paper, we explore the creation of adversarial triggers for ESMFold to explore the robustness of the protein folding model.

2 Algorithm

Our algorithm essentially consists of replicating the Universal Adversarial Trigger algorithm by Wallace et al.[13], itself inspired by HotFlip,[5] for the ESMFold protein model. However, there are some key differences, notably in the loss function. What follows is an overview of the universal adversarial trigger algorithm with our modifications highlighted.

We have a model f which takes a text input $\mathbf{t}$ of n residues in FASTA format. The model has one output of particular importance: the three-dimensional position of each central alpha-carbon (Cα) in the predicted protein ($\mathbb{R}^{n\times 3}$).

After the FASTA input $\mathbf{t}$, we concatenate an adversarial trigger $\mathbf{t}_{adv}$ with a fixed length m. Our objective becomes the following:

$$\arg\min_{\mathbf{t}_{adv}} \mathbb{E}_{p\in\mathcal{P}} \left[\mathcal{L}\left(\tilde{y}_p, f(\mathbf{t}_p; \mathbf{t}_{adv})\right)\right], \tag{1}$$

where $\mathcal{L}$ is a loss function, $\mathcal{P}$ are real proteins, $\mathbf{t}_p$ is the FASTA sequence for protein p, and $\tilde{y}_p$ is a target 3D structure for protein p.

Rather than iterating through all 20^m possible values of $\mathbf{t}_{adv}$ to find the one that minimizes $\mathbb{E}_{p\in\mathcal{P}}\mathcal{L}$, we update embeddings in order to minimize the loss's first-order Taylor expansion around the embedding $\mathbf{e}_{adv_i}$ of the current residue:

$$\mathbf{e}_{adv_i} \leftarrow \arg\min_{\mathbf{e}'_i\in\mathcal{V}} \left[\mathbf{e}'_i - \mathbf{e}_{adv_i}\right]^\top \nabla_{\mathbf{e}_{adv_i}}\mathcal{L}, \tag{2}$$

where $\mathcal{V}$ is the set of all 20 token embeddings for each residue and $\nabla_{\mathbf{e}_{adv_i}}\mathcal{L}$ is the average gradient of $\mathcal{L}$ with respect to the embedding $\mathbf{e}_{adv_i}$ over a batch. After finding each new $\mathbf{e}_{adv_i}$, we convert the embedding back to a token at the ith position of $\mathbf{t}_{adv}$.

However, to improve stability, we do not apply this change to tokens at all positions. Instead, we apply the change only to those 5 tokens with the smallest value of $\left[\mathbf{e}'_i - \mathbf{e}_{adv_i}\right]^\top \nabla_{\mathbf{e}_{adv_i}}\mathcal{L}$. Additionally, if

the loss has been greater than the minimum loss thus far for three consecutive steps, we do not apply equation (2). In such a case, we reset the trigger to the previously found trigger with the lowest loss and then change three residues at random.

In general, the predicted and true coordinates of each residue will differ up to a rotation and a translation, even if the structures themselves are quite similar in 3D space. Therefore, we use the Kabsch-Umeyama algorithm[8] to find the optimal translation and rotation and align the predicted and true structures before computing RMSD. This algorithm is detailed in appendix A.

3 Computing Triggers for ESMFold

We ran this algorithm in three settings, each with a different choice of $\mathcal{L}$ and $\mathcal{P}$:

1. We find a trigger to *maximize* the expected root mean square deviation (RMSD) between the predicted structure $y = f(\mathbf{t}; \mathbf{t}_{adv})$ and the true structure $\tilde{y}_p$ over all given proteins $p \in \mathcal{P}$, using only the first n predicted residues. (That is, we exclude the predicted positions of the residues in the trigger itself, and compare only the common residues.) We compute RMSD between the central alpha-carbon (Cα) atoms only. The true structures $\tilde{y}_p$ and sequences $\mathbf{t}_p$ were obtained through NMR via the RCSB database.[2]

2. Identical to the previous case, but we instead find a trigger to *minimize* the RMSD.

3. We limit $\mathcal{P}$ to one particular protein and set $\mathbf{t}_p$ to an empty sequence. Then we find $\mathbf{t}_{adv}$ to minimize the RMSD between the predicted structure y and the true structure $\tilde{y}_p$ over all residues, effectively reconstructing the protein's sequence from its structure.

In the first two cases, we initialize $\mathbf{t}_{adv}$ with a string of glycines of the appropriate length, and we repeat the step given by (2) 16 times. Glycine was chosen because the original universal adversarial trigger used filler tokens like "the" and "a" for initialization,[13] and glycine is the simplest amino acid. In the third case, we initialize $\mathbf{t}_{adv}$ with a random sequence and run the algorithm for 256 steps.

3.1 Maximize RMSD

Our first experiment finds a trigger that maximizes the mean RMSD between the true and predicted structures over a set of real proteins. This is the closest case to a true "adversarial" attack: we aim to find a trigger that disturbs the model's predictions as much as possible. The trigger was initialized as 10 glycines.

We used 19 proteins sourced from across *E. coli*, *S. cerevisiae* (baker's yeast), *M. musculus* (mouse), and *H. sapiens* (human). Each protein was 90 residues or fewer in length. We chose these small numbers due to computational constraints and because of the similarly small numbers used with success in the universal adversarial trigger algorithm (thirty manually written tweets with average length 10 tokens).[13] We used Uniprot[11] to find these proteins and Biopython[3] to parse PDB structure files from RCSB[2].

In this case, our loss function is given by

$$\mathcal{L} = -\frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \mathrm{RMSD}\left(\tilde{y}_p, f(\mathbf{t}_p; \mathbf{t}_{adv})\right), \tag{3}$$

where RMSD is root mean square deviation between the structures after applying the Kabsch-Umeyama algorithm. Thus, by minimizing loss, we are maximizing RMSD.

The initial mean RMSD between the model's predictions and the ground truth structures was recorded across all proteins. This RMSD was compared to the mean RMSD at each step in the trigger algorithm, as shown in figure 1. In the end, the best trigger was found to be PPPKHGKHPH (FASTA), with a mean RMSD of 25.7 Å compared to 24.9 Å with no trigger added.

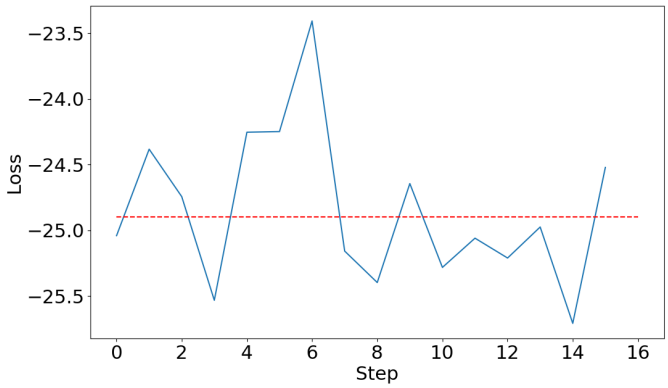
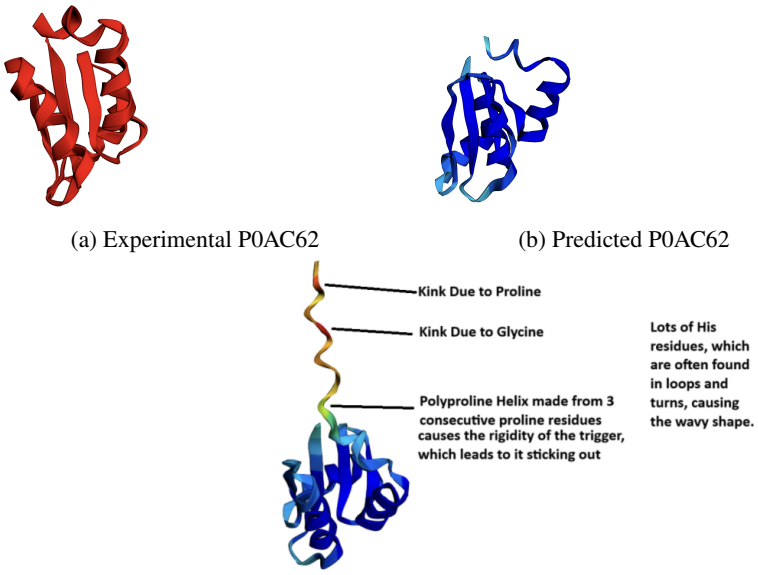


Figure 1: Loss (given by equation (3)) in Angstroms for maximizing RMSD over each step. The red line represents the mean negative RMSD in Angstroms between the model's predictions and the true structures without any trigger added.

Based on figure 1, there is no clear trend: the trigger is unable to significantly increase the RMSD. (Note that the y-axis in the figure is the negative RMSD, as given by equation (3).)

A sample protein structure with the trigger added is shown in figure 2.



(a) Experimental P0AC62

(b) Predicted P0AC62

(c) Predicted P0AC62 with trigger added and explanation of amino acid effects

Figure 2: Predicted and experimental structures for *E. coli* glutaredoxin 3 (P0AC62) with the trigger from experiment 3.1. Color of predicted sequences represents the model's predicted uncertainty.

3.2 Minimize RMSD

This experiment is identical to the one described in section 3.1, except that our loss function is given by the negative of equation (3). Thus, by minimizing loss, we are minimizing RMSD.

As in section 3.1, we compare our results to the mean RMSD between the protein model predictions and the true structures (24.9 Å). We show the loss at each step in figure 3. In the end, the best trigger was found to be GCWEKYDYSQ (FASTA), with a mean RMSD of 23.0 Å compared to 24.9 Å with no trigger added.

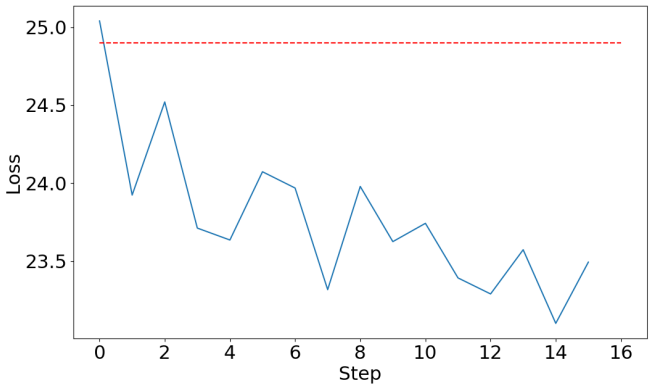


Figure 3: Loss (given by the negative of equation (3)) in Angstroms for minimizing RMSD over each step. The red line represents the mean RMSD in Angstroms between the model's predictions and the true structures without any trigger added.

Because these results were relatively promising, we expanded them to 64 steps, as shown in figure 4. This achieved a minimum RMSD of 21.2 Å, but eventually reached a plateau.

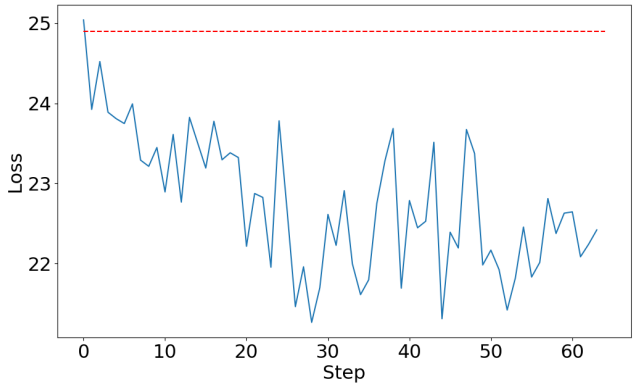


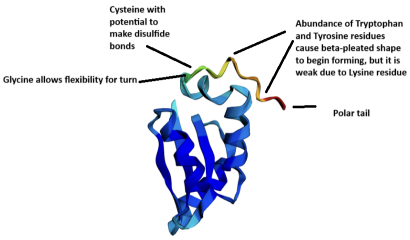
Figure 4: Loss (given by the negative of equation (3)) in Angstroms for minimizing RMSD over each step. The red line represents the mean RMSD in Angstroms between the model's predictions and the true structures without any trigger added.

Unlike the previous section, figures 3 and 4, show a clear trend: the trigger is able to significantly decrease the RMSD, even below the level seen with no trigger. Logically, this is a consequence of a model inaccuracy: it does not make much sense for adding a trigger to make the model more accurately predict proteins than using their true unmodified structures.

A sample protein structure with the trigger added is shown in figure 5.

(a) Experimental P0AC62

(b) Predicted P0AC62



(c) Predicted P0AC62 with trigger added and explanation of amino acid effects

Figure 5: Predicted and experimental structures for *E. coli* glutaredoxin 3 (P0AC62) with the trigger from experiment 3.2. Color of predicted sequences represents the model's predicted uncertainty.

3.3 Reconstruct Protein Sequence

In this experiment, our loss is given by

$$\mathcal{L} = \text{RMSD}\left(\tilde{y}_{p_0}, f(\mathbf{t}_{adv})\right) \tag{4}$$

for some fixed $p_0 \in \mathcal{P}$. Here $\mathbf{t}_{adv}$ is the entire model input, not a trigger concatenated to the end of another sequence. That is, we are finding the sequence $\mathbf{t}_{adv}$ that minimizes the RMSD between the structure $\tilde{y}_{p_0}$ of protein p and the model output $f(\mathbf{t}_{adv})$. Ideally, given an accurate model f, this task is equivalent to finding the sequence $\mathbf{t}_{adv}$ of a protein p_0 given its structure $\tilde{y}_{p_0}$.

We run this algorithm on the protein P0AC62, shown in figures 2 and 5. In figure 6, we graph the loss at each step as well as the Levenshtein distance between the "trigger" at that point and the true FASTA sequence of the protein.

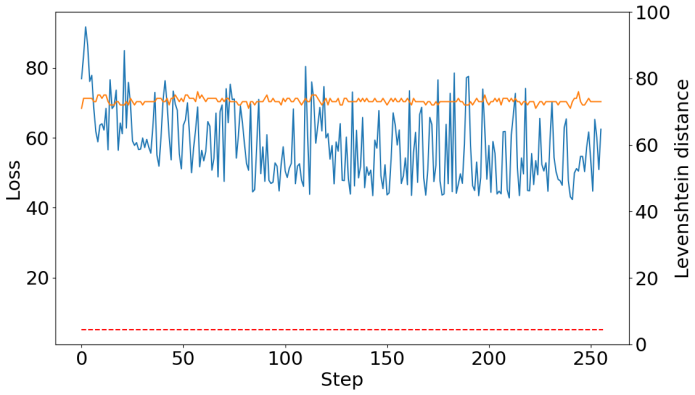


Figure 6: In blue: loss (given by equation (4)) in Angstroms between the predicted structure and the true structure for protein $p_0 =$ P0AC62. Red dashed line: RMSD between predicted and true structures for P0AC62. In orange: Levenshtein distance between the predicted sequence and the true sequence at each step.

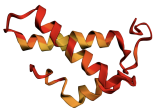
In figure 6, there is not much of a trend towards decreasing loss. The loss never approaches the red line (difference between true and predicted structures with the correct sequence), and the Levenshtein

distance remains high and fairly constant throughout. The RMSD between the predicted and true structures for P0AC62 is 5.2 Å, while the lowest RMSD achieved by the algorithm is 42.4 Å.

The optimal structure discovered by our algorithm (with RMSD 42.4 Å) is shown in figure 7. In this figure, we see the presence of alpha helices in both the trigger algorithm sequence and the ground truth, which could either be chance or could represent the model partially learning the sequence. Compared to the prediction from the correct sequence, the model is highly uncertain about its prediction through the trigger algorithm, which may be a consequence of this computationally-generated sequence being out of distribution for the model.

(a) Experimental P0AC62

(b) Predicted P0AC62 with correct sequence



(c) Predicted structure from trigger algorithm sequence

Figure 7: Predicted and experimental structures for *E. coli* glutaredoxin 3 (P0AC62) with the trigger from experiment 3.3. Color of predicted sequences represents the model's predicted uncertainty.

4 Conclusion

4.1 Analysis

In section 3.1, we attempted to attach a trigger to proteins in order to increase the RMSD between predicted and ground truth protein structures. This was largely unsuccessful, as can be seen in figure 1. There was no consistent trend towards higher RMSD by applying the algorithm step given by equation (2).

However, section 3.2 had slightly better results, shown in figure 3. We found that adding a particular trigger to our set of proteins actually *decreased* the RMSD between predicted and ground truth structures, even though the trigger does not exist in the true structure. This possible indicates a model vulnerability: the model may underestimate some factor in the folding of these proteins, and it may be that the trigger given in section 3.2 represents a compensation for this deficiency in the model. More testing across a wider range of proteins would be needed to determine if this trigger is truly universal, in the case of natural language processing.[13]

In section 3.3, we attempted to apply this algorithm to reconstruct a protein sequence given its structure. There was no consistent decrease in RMSD from applying the algorithm, and the Levenshtein distance between the true and predicted structures remained high throughout, as shown in figure 6.

4.2 Applications

Overall, this algorithm was unsuccessful in discovering the desired triggers. However, if these triggers exist and could be find, there are a number of important applications that would result.

By identifying specific triggers, we can learn more about both the underlying mechanism of how these protein folding models decide on structures and more about the nature of protein folding in the real world. We can learn more about what parts of the input are more prominent in the models architecture for determining structure and see if the sequences that show up are physically feasible to cause as much disruption as they do in predictions. This can help find vulnerabilities in the models or

particular sequences that the models are more responsive to than others. This is especially important because models like these are used in synthetic biology to create drugs which have the potential to save lives, and thus accuracy in what these drugs will do in the body is critical. By further studying these triggers, researchers can ensure that the models stay reliable even when tasked with novel sequences, which will improve performance in real world applications.

In a similar vein, this approach can be used to identify universal triggers, which are triggers that produce a similar change across multiple different proteins. If these exist, we would learn more about conserved folding mechanisms or structural relations in proteins or learn more about how the biases in the model consistently interpret small patterns in the data. Knowledge of universal triggers can also help improve models by preventing an overreaction on certain patterns or correlations in the input and make them focus more on replicating the physical principles underlying protein folding. This would ensure the models mirror real-world protein folding as much as possible.

Further, these triggers can be used in the opposite direction to facilitate improved drugs. An early attempt in this direction is given in section 3.3. Expanding on this, we can select for beneficial structural changes, such as configurations that help a drug last longer in low pH conditions such as the stomach. This can greatly increase the types of drugs that can be given orally instead of via IV or injection, which is a much better patient experience. We can also try to search for triggers that improve protein stability, which will lead to drugs that last longer in the human body without the need for frequent boosters or doses to ensure effectiveness. Similarly, if triggers can be used to find versions of common drugs that last longer in vivo, it would prevent the constant need to retake drugs for patients.

We can also add triggers to improve protein efficacy. For example, if we add a trigger that changes the folding of a protein leading to the active site being more likely to bind, we can greatly decrease the K_M (Michaelis constant) of the protein, which indicates that it is more likely to bind to the substrate and thus more effective. For patients with loss of function mutations leading to impaired protein function, this can help discover analogs that can make up for the deficient natural protein.

4.3 Future Research

Although this algorithm does not directly translate well from natural processing to protein folding, its incredible success in the natural language processing domain is suggestive of future research. Further modifications could greatly increase its efficacy in this new biological domain.

Firstly, the original universal attack trigger algorithm included beam search on top of the step given in equation (2). Since the vocabulary size of residues is quite small (only 20), it is possible that beam search will search a relatively large portion of the space of triggers and have an even greater impact than was observed in NLP. Other modifications to the core step of the algorithm, such as a switch to projected gradient descent,[10] may also be helpful.

Second, it can be seen from figure 7 that in the protein reconstruction application of this algorithm, the model's confidence in its own prediction is very low. Assuming this indicates an extremely out-of-distribution sequence, we may fix this by restricting the search space to sequences that are more likely to be in distribution. For example, rather than working at the level of individual residues, we may use a larger vocabulary whose tokens are some of the most common small sequence components observed in a dataset of protein sequences. This may allow the algorithm to work on the scale of common secondary structure components such as alpha helices rather than exploring sequences that are unlikely to appear in nature.

Finally, future research can simply scale up this algorithm. We used only 19 sequences in sections 3.1 and 3.2, each one under 90 residues, which is fairly small for a protein. By using more GPUs, we may be able to consider both more distinct sequences, longer sequences, and longer triggers, which may lead to new insights that were missing in this relatively small dataset. This is supported by the original universal adversarial triggers paper, where longer triggers were generally more effective.[13]

References

[1] Christian B. Anfinsen. Principles that govern the folding of protein chains. *Science*, 181(4096):223–230, 1973.

[2] Helen M. Berman, John Westbrook, Zukang Feng, Gary Gilliland, T. N. Bhat, Helge Weissig, Ilya N. Shindyalov, and Philip E. Bourne. The protein data bank. *Nucleic Acids Research*, 28(1):235–242, 01 2000.

[3] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cymon J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 03 2009.

[4] Ken A. Dill, S. Banu Ozkan, M. Scott Shell, and Thomas R. Weikl. The protein folding problem, 2008.

[5] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. Hotflip: White-box adversarial examples for text classification, 2018.

[6] Peter L. Freddolino, Christopher B. Harrison, Yanxin Liu, and Klaus Schulten. Challenges in protein folding simulations: Timescale, representation, and analysis, 2010.

[7] John M. Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A A Kohl, Andy Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596:583 – 589, 2021.

[8] Jim Lawrence, Javier Bernal, and Christoph Witzgall. A purely algebraic justification of the kabsch-umeyama algorithm. *Journal of Research of the National Institute of Standards and Technology*, 124, October 2019.

[9] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Sal Candido, et al. Language models of protein sequences at the scale of evolution enable accurate structure prediction. *bioRxiv*, 2022.

[10] Nicolas Papernot, Patrick McDaniel, Ananthram Swami, and Richard Harang. Crafting adversarial input sequences for recurrent neural networks, 2016.

[11] The UniProt Consortium. Uniprot: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, page gkae1010, 11 2024.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[13] Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. Universal adversarial triggers for attacking and analyzing nlp, 2021.

[14] Robert Zwanzig, Attila Szabo, and Bidman Bagchi. Levinthal's paradox., 1992.

A Kabsch-Umeyama Algorithm

In general, the predicted and true coordinates of each residue will differ up to a rotation and a translation, even if the structures themselves are quite similar in 3D space. Therefore, we use the Kabsch-Umeyama algorithm[8] to find the optimal translation and rotation and align the predicted and true structures before computing RMSD.

The steps of the Kabsch-Umeyama algorithm are as follows:[8]

1. Given two sets of n points expressed as matrices $Q \in \mathbb{R}^{3\times n}$ and $P \in \mathbb{R}^{3\times n}$, compute the 3×3 matrix $M = QP^T$.
2. Compute the singular value decomposition (SVD) of M, $M = USV^T$.
3. Compute the diagonal matrix
$$\tilde{S} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & s_3 \end{bmatrix},$$
where $s_3 = 1$ if $\det(VU) > 0$, -1 otherwise.
4. The optimal rotation matrix is given as $R = V\tilde{S}U^T$.

By centering Q and P on the origin and then applying the rotation matrix R to Q, we minimize the RMSD between Q and P. In a sense, we place both structures in the same coordinate system, and can now compare them directly. Before applying the above algorithm to find the optimal rotation matrix, we recenter each protein on its average Cα atomic coordinate.

B Project Use

Some code for this project was used by me (Harys) for my biological physics class. However, all algorithms and results are specific to this paper. No part of this paper's body was or will be used for another course.